Predictive Modelling of Hotel Room Rates (ADR) Using Machine Learning

Project Report

Table of Contents

1. Executive Summary: 3
2. Introduction: 3
   2.1 Business Problem & Objectives: 3
   2.2 Scope & Success Criteria 4
3. Data Description: 4
   3.1 Dataset Source & Granularity: 4
   3.2 Data Loading and Summary Statistics: 5
   3.3 Summary Statistics of Key Variables: 6
   3.4 Key Variables Description: 6
   3.5 Key Observations: 7
4. Data wrangling: 8
   4.1 Handling Missing Values: 8
   4.2 Data Types: 9
   4.3 Outliers: 10
   4.4 Integrity: 10
   4.5 Text Hygiene for Categorical Features: 11
   4.6 Encoding: 12
   4.7 Feature Engineering and Transformation: 13
   4.8 Data Normalization: 14
5. Exploratory Data Analysis (EDA): 15
6. Regression Modelling: 19
   6.1 Objective: 19
   6.2 Linear Regression: 19
   6.3 Decision Tree Regressor 20
   6.4 Random Forest Regressor 21
   6.5 Hyperparameter Tuning and Cross-Validation (Random Forest Regressor): 22
   6.6 Hyperparameter Tuning and Cross-Validation (Decision Tree Regressor): 24
   6.7 Model Comparison and Evaluation: 26
7. Business Insights: 26
8. Conclusion: 27

1. Executive Summary:

The goal of this project is to predict the Average Daily Rate (ADR) in the hospitality sector utilizing a real dataset that includes customer, booking, and market information. This is designed to assist revenue management within the hotel market that relies on data to inform pricing strategy based on demand conditions, customer segments, and the competitive environment.

The data includes data on more than 6,000 booking records, across 26 variables which represent varying characteristics related to aspects including, but not limited to, country, room accommodation type, booking channel, meals included, payment type, customer demographic characteristics and market pressure variables which include Event Distance Interaction, Demand Pressure and Customer Value Index. An initial audit identified minimal missing values, which were handled using mean and mode imputations. Outliers in ADR were partially retained, as they represent genuine high-value nights in the market.

The analysis firstly conducted exploratory data analysis (EDA) to understand distribution patterns and relationships between all variables next, it conducted predictive modelling with a Linear Regression, Random Forest, and Decision Tree approach. The models were then compared through a hold-out test set on R^2, MAE, and RMSE metrics from the test set and did so without splitting any of the data into a training or validation set. In this circumstance, the Random Forest model was able to achieve the highest accuracy with the lowest error by being able to capture complex interactions that exist between customer behaviour and demand dynamics.

A few high-level drivers of the ADR were found to be competition pricing, property occupancy rate, demand pressure, and customer value index in that order. One of the important details to point out is the exploratory performance by the models suggests there is an opportunity for hotels to implement context-aware pricing surrounding events, weekends, and high-occupancy.

Although the performance was strong, and the predictive performance was adequate, limitations still exist because of the cross-sectional nature of the data and the lack of time-series theories. Future versions of ADR based price modelling would likely require seasonality, event calendars, and long-term booking trends for increased precision.

2. Introduction:

2.1 Business Problem & Objectives:

Determining the right room rate (ADR) is a key challenge for revenue managers in the competitive environment of the hospitality market. Room rates need to be appealing to the guest while maximizing occupancy level and profit margin. When considering seasonal demand, competitor prices, customer segments, events, and source of business channels, manually establishing pricing will become cumbersome, even inefficient as a result.

This project aims to build a machine learning model for accurately predicting ADR utilizing both operational (e.g., room type, occupancy, booking channel) and market (e.g., demand pressure, event distance) variables. The predictive model provides supports for data-based decision-making related to:

- Dynamic pricing that aligns with market demand and competitive prices.
- More accurate revenue forecasting and financial planning.
- Improved customer segmentation and promotions.
- Increased transparency to understand what affects ADR.

Ultimately, the end game of the model is to maximize both profit and guest satisfaction leveraging AI-based insight, moving hotels from a reactive approach to a proactive revenue management approach.

2.2 Scope & Success Criteria

The scope of this project includes:

- Using a **hospitality dataset** with more than 6,000 observations and 26 features.
- Conducting **data wrangling, exploratory data analysis (EDA), feature engineering, and regression modelling** as required.
- Implementing and evaluating **Linear Regression**, **Random Forest**, and **Decision Tree** models based on their predictive capability.

Success will be evaluated based on:

1. **Model performance** — low Mean Absolute Error (MAE) and Root Mean Square Error (RMSE).
2. **Interpretability** — clear identification of key ADR drivers.
3. **Business Relevance** — insights that translate into practical pricing strategies.

3. Data Description:

3.1 Dataset Source & Granularity:

The dataset used in this project originates from a hospitality management system, and consists of authentic hotel bookings and performance data capturing customer, booking, property, and market dimensions. Each record corresponds to one individual hotel stay, which indicates the data is relatively cross-sectional as opposed to time-series. This granularity allows analyses of hotel bookings at the individual level that are appropriate for forecasting nightly Average Daily Rate (ADR) based on customer context and "shocks" or demand conditions occurring outside of the hotel's control.

3.2 Data Loading and Summary Statistics:

```python
# Basic Info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6630 entries, 0 to 6629
Data columns (total 26 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Booking_ID               6630 non-null   int64
 1   Hotel_ID                 6630 non-null   object
 2   Country                  6529 non-null   object
 3   Brand_Tier               6630 non-null   object
 4   Star_Rating              6630 non-null   int64
 5   Room_Type                6630 non-null   object
 6   Nights_Stayed            6630 non-null   int64
 7   Booking_Channel          6630 non-null   object
 8   Meals_Included           6472 non-null   object
 9   Payment_Type             6630 non-null   object
 10  Customer_Age             6368 non-null   float64
 11  Lead_Time_Days           6630 non-null   int64
 12  Checkin_Date             6630 non-null   object
 13  Checkin_Month            6630 non-null   int64
 14  Is_Weekend               6630 non-null   int64
 15  Is_Public_Holiday        6630 non-null   int64
 16  Occupancy_Rate_Property  6562 non-null   float64
 17  Competitor_Avg_Rate_RM   6447 non-null   float64
 18  Distance_To_Center_KM    6630 non-null   float64
 19  Review_Score             6489 non-null   float64
 20  Promo_Applied            6630 non-null   int64
 21  Events_Index_City        6490 non-null   float64
 22  ADR_RM                   6599 non-null   float64
 23  Total_Spend_RM           6630 non-null   float64
 24  Extra_Spend_Unscaled     6630 non-null   int64
 25  Satisfaction_Rating      6630 non-null   int64
dtypes: float64(8), int64(10), object(8)
memory usage: 1.3+ MB
```

The dataset was imported from the file **Hospitality_Wrangling_EDA_Regression.csv** using the Pandas library. After loading, basic checks such as **.head ()**, **.info()**, and **.shape** confirmed that the dataset contained: **6,630 rows and 26 columns** before cleaning, each row representing an individual hotel booking record. The key variables in the dataset include:

- **Target Variable:** ADR_RM – Average Daily Rate (in RM), representing the price per room per night.
- **Customer & Booking Features:** Country, Customer_Age, Booking_Channel, Payment_Type, Meals_Included
- **Property & Operational Features:** Room_Type, Occupancy_Rate_Property, Brand_Tier, Review_Score
- **Market & Contextual Features:** Competitor_Avg_Rate_RM, Occupancy_Rate_Property, Events_Index_City.

  These variables jointly capture the **supply–demand dynamics** and **customer heterogeneity** that drive hotel pricing decisions.

3.3 Summary Statistics of Key Variables:

A summary of numerical fields was generated to understand the overall data distribution.

```python
# Summary statistics (mean, median, std) for numeric columns

numeric_cols = df.select_dtypes(include=['number']).columns.tolist()

summary = df[numeric_cols].describe().T  # contains mean, std, min, 25%, 50%, 75%, max
summary['median'] = df[numeric_cols].median()
summary = summary[['mean', 'median', 'std', 'min', '25%', '50%', '75%', 'max', 'count']]
summary = summary.rename(columns={'count':'n_nonnull'})
summary.style.format("{:.3f}")
```

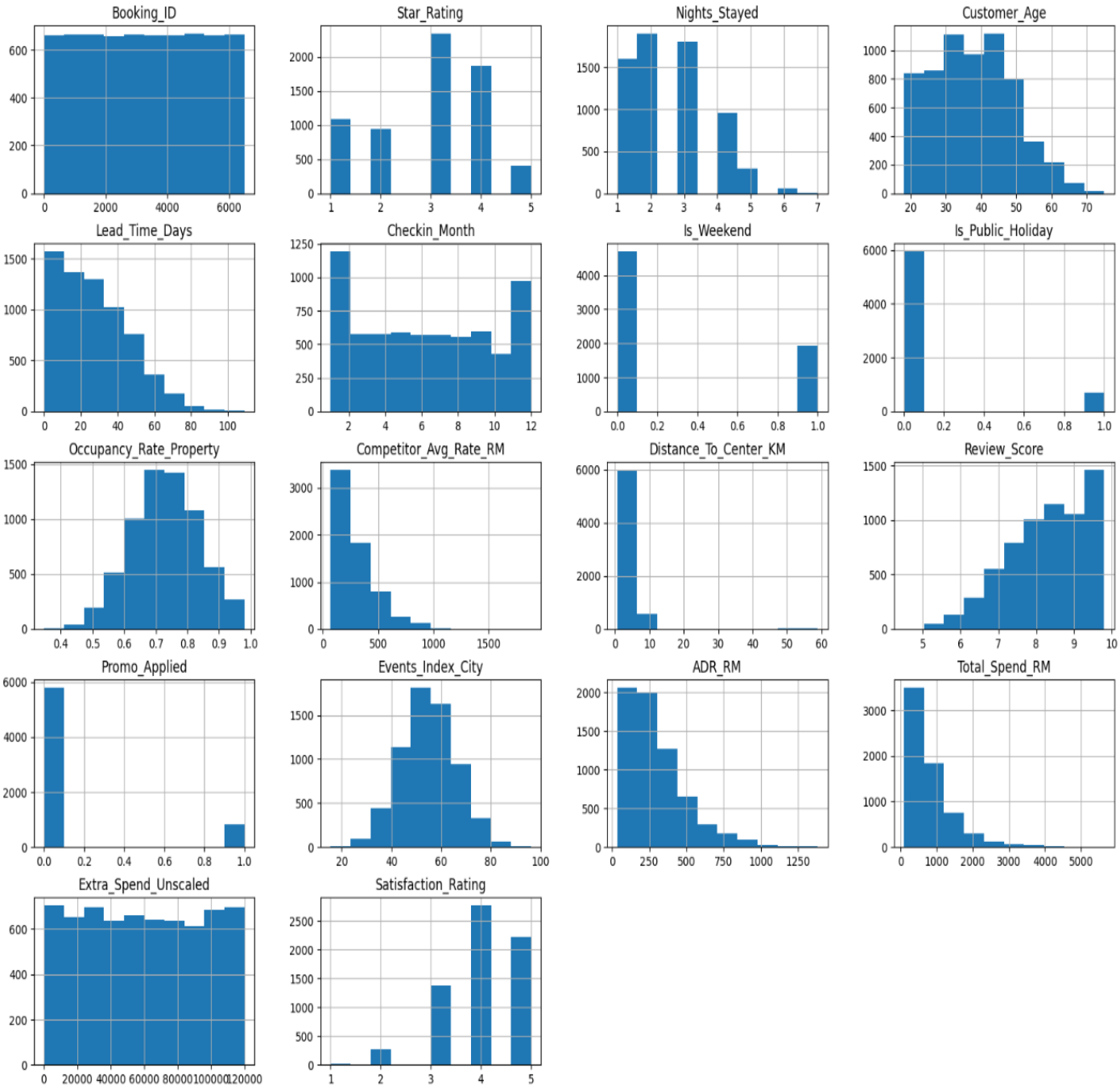
	mean	median	std	min	25%	50%	75%	max	n_nonnull
Booking_ID	3250.912	3249.500	1877.482	1.000	1621.250	3249.500	4876.750	6500.000	6630.000
Star_Rating	2.937	3.000	1.148	1.000	2.000	3.000	4.000	5.000	6630.000
Nights_Stayed	2.497	2.000	1.192	1.000	2.000	2.000	3.000	7.000	6630.000
Customer_Age	37.611	37.000	11.641	18.000	29.000	37.000	46.000	75.000	6368.000
Lead_Time_Days	26.856	25.000	19.519	0.000	11.000	25.000	40.000	109.000	6630.000
Checkin_Month	6.244	6.000	3.413	1.000	3.000	6.000	9.000	12.000	6630.000
Is_Weekend	0.292	0.000	0.455	0.000	0.000	0.000	1.000	1.000	6630.000
Is_Public_Holiday	0.103	0.000	0.303	0.000	0.000	0.000	0.000	1.000	6630.000
Occupancy_Rate_Property	0.729	0.729	0.107	0.347	0.656	0.729	0.804	0.980	6562.000
Competitor_Avg_Rate_RM	288.430	242.820	193.514	70.000	128.960	242.820	391.260	1884.820	6447.000
Distance_To_Center_KM	4.324	3.850	4.199	0.650	2.800	3.850	5.550	58.871	6630.000
Review_Score	8.306	8.400	1.075	4.500	7.550	8.400	9.180	9.800	6489.000
Promo_Applied	0.124	0.000	0.330	0.000	0.000	0.000	0.000	1.000	6630.000
Events_Index_City	54.875	54.700	11.155	15.500	47.300	54.700	62.400	96.200	6490.000
ADR_RM	294.418	245.280	200.268	35.000	131.820	245.280	397.945	1381.420	6599.000
Total_Spend_RM	803.040	597.150	652.927	85.660	351.690	597.150	1040.710	5644.460	6630.000
Extra_Spend_Unscaled	59830.687	59329.500	34996.892	136.000	29499.750	59329.500	90485.000	119999.000	6630.000
Satisfaction_Rating	4.042	4.000	0.846	1.000	4.000	4.000	5.000	5.000	6630.000

3.4 Key Variables Description:

Variable	Description
ADR_RM	Target variable; represents the average rate charged per night per room.
Competitor_Avg_Rate_RM	This is the average price in the market among competitors that we will use to assess whether the rates are competitive.
Total_Spend_RM	The amount of total spends per booking which consists of room, meals and extras.
Occupancy_Rate_Property	The percentage of rooms occupied; this is an indicator of demand for the property and operational strain.
Customer_Age	The age of the customer; this will assist in understanding demographic trends in pricing.
Lead_Time_Days	The number of days there are from booking until check in day; this will allow us to understand early booking and last-minute booking behaviour.
Nights_Stayed	The number of nights stayed with the booking with longer stays having potential perceptions of lower ADR.
Events_Index_City	Captures event intensity in the city; higher values correspond to busy event periods.

Distance_To_Center_KM	Hotel's distance from the city center; affects location-based pricing.
Review_Score	The average customer rating from reviews for the property.
Star_Rating	The category of the property; higher star ratings may correspond to higher premium brands.

3.5 Key Observations:



- Average Daily Rate (ADR), competitive rates and total spend appear to be positively skewed, indicating a small proportion of higher-end stays.
- The average occupancy level is approximately 69%, which is consistent with typical operations in an urban hotel market.
- The customers' ages are mainly around the 30-40 age category, confirming a relatively balanced representation across demographics.
- Lead time in the data shows a broad variation (0-90 days) for different booking behaviours from last-minute to early planners.

- Event index and demand pressure variables suggest that external market factors are embedded in the dataset — crucial for pricing prediction.

4. Data wrangling:

4.1 Handling Missing Values:

```
# Checking Missing Values
df_clean.isnull().sum()
```

	0
Booking_ID	0
Hotel_ID	0
Country	101
Brand_Tier	0
Star_Rating	0
Room_Type	0
Nights_Stayed	0
Booking_Channel	0
Meals_Included	158
Payment_Type	0
Customer_Age	261
Lead_Time_Days	0
Checkin_Date	0
Checkin_Month	0
Is_Weekend	0
Is_Public_Holiday	0
Occupancy_Rate_Property	68
Competitor_Avg_Rate_RM	182
Distance_To_Center_KM	0
Review_Score	141
Promo_Applied	0
Events_Index_City	140
ADR_RM	31
Total_Spend_RM	0
Extra_Spend_Unscaled	0
Satisfaction_Rating	0

dtype: int64

Before Cleaning

```
# Adjusting Missing Values

df_clean['Country'].replace(np.nan, df_clean['Country'].mode()[0], inplace=True)
df_clean['Meals_Included'].replace(np.nan, df_clean['Meals_Included'].mode()[0], inplace=True)
df_clean['Customer_Age'].replace(np.nan, df_clean['Customer_Age'].mean(), inplace=True)
df_clean['Occupancy_Rate_Property'].replace(np.nan, df_clean['Occupancy_Rate_Property'].mean(), inplace=True
df_clean['Competitor_Avg_Rate_RM'].replace(np.nan, df_clean['Competitor_Avg_Rate_RM'].mean(), inplace=True)
df_clean['Review_Score'].replace(np.nan, df_clean['Review_Score'].mean(), inplace=True)
df_clean['Events_Index_City'].replace(np.nan, df_clean['Events_Index_City'].mean(), inplace=True)
df_clean['ADR_RM'].replace(np.nan, df_clean['ADR_RM'].mean(), inplace=True)
df_clean.isnull().sum()
```

After Cleaning

	0
Booking_ID	0
Hotel_ID	0
Country	0
Brand_Tier	0
Star_Rating	0
Room_Type	0
Nights_Stayed	0
Booking_Channel	0
Meals_Included	0
Payment_Type	0
Customer_Age	0
Lead_Time_Days	0
Checkin_Date	0
Checkin_Month	0
Is_Weekend	0
Is_Public_Holiday	0
Occupancy_Rate_Property	0
Competitor_Avg_Rate_RM	0
Distance_To_Center_KM	0
Review_Score	0
Promo_Applied	0
Events_Index_City	0
ADR_RM	0
Total_Spend_RM	0
Extra_Spend_Unscaled	0
Satisfaction_Rating	0

dtype: int64

Prior to data cleansing, select attributes within the dataset contained missing values, including the following: Meals_Included (158), Payment_Type (261), Customer_Age (101), Review_Score (141) and ADR_RM (31). Missing observations can introduce bias to statistical analysis and can

impair model quality and performance; we thus undertook suitable imputation for replacement based on the pertinent type. For categorical variables such as Country and Meals_Included, we employed the mode (most common frequency) in order to replace the missing entries. This preserves the most representative category of the data without impact to the overall distribution of categorical variables. For all other numerical variables such as Customer_Age, Review_Score, Competitor_Avg_Rate_RM and ADR_RM, missing values were replaced by the mean of each column. The mean retains central tendency and limits data loss without disturbing the numerical balance. Post imputation, all missing values were thoroughly resolved, producing a complete and consistent dataset. This cleansing process ensures subsequent exploratory data analysis and machine learning models can be conducted without bias, distance or compute errors arising out of missing data.

4.2 Data Types:

```
# Basic Info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6630 entries, 0 to 6629
Data columns (total 26 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Booking_ID                 6630 non-null   int64
 1   Hotel_ID                   6630 non-null   object
 2   Country                    6529 non-null   object
 3   Brand_Tier                 6630 non-null   object
 4   Star_Rating                6630 non-null   int64
 5   Room_Type                  6630 non-null   object
 6   Nights_Stayed              6630 non-null   int64
 7   Booking_Channel            6630 non-null   object
 8   Meals_Included             6472 non-null   object
 9   Payment_Type               6630 non-null   object
 10  Customer_Age               6368 non-null   float64
 11  Lead_Time_Days             6630 non-null   int64
 12  Checkin_Date               6630 non-null   object
 13  Checkin_Month              6630 non-null   int64
 14  Is_Weekend                 6630 non-null   int64
 15  Is_Public_Holiday          6630 non-null   int64
 16  Occupancy_Rate_Property    6562 non-null   float64
 17  Competitor_Avg_Rate_RM     6447 non-null   float64
 18  Distance_To_Center_KM      6630 non-null   float64
 19  Review_Score               6489 non-null   float64
 20  Promo_Applied              6630 non-null   int64
 21  Events_Index_City          6490 non-null   float64
 22  ADR_RM                     6599 non-null   float64
 23  Total_Spend_RM             6630 non-null   float64
 24  Extra_Spend_Unscaled       6630 non-null   int64
 25  Satisfaction_Rating        6630 non-null   int64
dtypes: float64(8), int64(10), object(8)
memory usage: 1.3+ MB
```

The dataset includes both categorical and numerical variables. Categorical variables, such as Country, Brand_Tier, Room_Type, Booking_Channel, and Payment_Type are encoded as object data types; these will represent the text- based categories that will be used in the grouping and encoding of variables. Numerical variables can be classified as int64 or float64. Int64 is to discrete numerical variables such as Nights_Stayed, Lead_Time_Days, or Is_Public_Holiday, and float64 is used for continuous metrics such as Customer_Age, Occupancy_Rate_Property, Competitor_Avg_Rate_RM, ADR_RM, and Review_Score. The combination of present data types provides a good basis for subsequent statistical analysis and predictive modelling.

4.3 Outliers:

```python
# Identify outliers using IQR
numeric_cols=df_clean.select_dtypes(include=['number']).columns.tolist()

print("\nIdentifying outliers using IQR:")
outlier_indices = {}
for col in numeric_cols:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df_clean[(df_clean[col] < lower_bound) | (df_clean[col] > upper_bound)].index
    outlier_indices[col] = outliers
    print(f"  {col:>20}: {len(outliers)} outliers identified")

# Combine indices from all columns
all_outlier_indices = [idx for indices in outlier_indices.values() for idx in indices]
all_outlier_indices = list(set(all_outlier_indices)) # Remove duplicates

print(f"\nTotal unique outliers identified across numeric columns: {len(all_outlier_indices)}")
```

```
Identifying outliers using IQR:
            Booking_ID: 0 outliers identified
           Star_Rating: 0 outliers identified
         Nights_Stayed: 364 outliers identified
          Customer_Age: 19 outliers identified
        Lead_Time_Days: 26 outliers identified
         Checkin_Month: 0 outliers identified
            Is_Weekend: 0 outliers identified
     Is_Public_Holiday: 676 outliers identified
  Occupancy_Rate_Property: 21 outliers identified
  Competitor_Avg_Rate_RM: 181 outliers identified
  Distance_To_Center_KM: 39 outliers identified
          Review_Score: 8 outliers identified
         Promo_Applied: 820 outliers identified
     Events_Index_City: 52 outliers identified
                ADR_RM: 185 outliers identified
        Total_Spend_RM: 343 outliers identified
  Extra_Spend_Unscaled: 0 outliers identified
   Satisfaction_Rating: 275 outliers identified

Total unique outliers identified across numeric columns: 2280
```

Outliers were identified using the **Interquartile Range (IQR) method** across all numeric columns. A total of **2,280 unique outliers** were detected, mainly within Promo_Applied (820), Is_Public_Holiday (676), Nights_Stayed (364), Total_Spend_RM (343), ADR_RM (185) and several others in smaller counts. After removing these records, the dataset reduced from **6,630 to 4,331 rows**, improving overall data reliability and model stability. Extreme values beyond realistic business ranges (e.g., extremely high spend or negative review scores) were considered errors rather than genuine premium cases.

4.4 Integrity:

To maintain data integrity and avoid redundancy therein the dataset duplicate records were detected. There were initially a total of 19 duplicates recorded based on potentially repeated booking entries or data entry mistakes. Duplicate observations result in biased statistical summaries and significantly shape models while providing disproportionate weight to observations. Accordingly, all duplicates were removed from the dataset. Following the deduplication process, no duplicates were

```python
# Checking for Duplicates
df.duplicated().sum()
```

```
np.int64(19)
```

```python
# Removing Duplicates
df_clean=df.drop_duplicates().reset_index(drop=True)

df_clean.shape
```

```
(6611, 26)
```

```python
# Checking for Duplicates After Cleaning
df_clean.duplicated().sum()
```

```
np.int64(0)
```

contained in the dataset, indicating the uniqueness of bookings and their integrity for following analyses, machine learning, and modelling.

Overall, the dataset provides a robust and realistic basis for building predictive models that reflect actual hotel revenue behaviour.

4.5 Text Hygiene for Categorical Features:

	Hotel_ID	Country	Brand_Tier	Room_Type	Booking_Channel	Meals_Included	Payment_Type	Checkin_Date
0	H006	Singapore	economy	Double	Walk-in	No	Online	2023-12-11
1	H071	Thailand	Midscale	Double	Online	Yes	Card	2023-10-02
2	H005	UK	economy	Single	Online	Yes	Online	2022-12-10
3	H074	Malaysia	economy	Double	Travel Agent	Yes	Card	2024-12-06
4	H066	THAILAND	midscale	Single	Online	Yes	Card	2022-11-07

```python
cat_cols = ["Country","Brand_Tier","Room_Type",
            "Booking_Channel","Meals_Included",
            "Payment_Type"]

cat_cols = [c for c in cat_cols if c in df_clean.columns]


df_clean[cat_cols] = (
    df_clean[cat_cols]
      .apply(lambda s: s.where(s.isna(), s.astype(str).str.strip().str.lower()))
      .replace(['nan', 'none', '', 'na', 'n/a'], np.nan)
)
```

```python
categoric_features_clean=df_clean.select_dtypes(include=['object'])

categoric_features_clean.head()
```

	Hotel_ID	Country	Brand_Tier	Room_Type	Booking_Channel	Meals_Included	Payment_Type	Checkin_Date
0	H006	singapore	economy	double	walk-in	no	online	2023-12-11
1	H071	thailand	midscale	double	online	yes	card	2023-10-02
2	H005	uk	economy	single	online	yes	online	2022-12-10
3	H074	malaysia	economy	double	travel agent	yes	card	2024-12-06
4	H066	thailand	midscale	single	online	yes	card	2022-11-07

To maintain consistency in categorical variables, all text-based columns were cleaned and standardized in terms of formatting. The categorical variables chosen for the study included Country, Brand_Tier, Room_Type, Booking_Channel, Meals_Included, and Payment_Type. In many categories, inconsistent casing of text and stray spaces were introduced in the columns or as strings within the categorical values (e.g., “Thailand” vs “thailand”, “nan”, “none”, “n/a”).

The textual cleaning happened in two primary steps: first, converting all unique text values to lowercase -- no changes are made to numeric data. Then, leading and trailing spaces were removed, as well as, any unrecognizable strings, and replaced them with NaN. This way, misformatted values could be dealt with later through a reconstruction where a value of NaN would allow for imputation. All this was accomplished via previously mentioned string manipulation functions worked with a lambda method and by using the replace() function. After processing, categorical text values have a consistent format (e.g., “Thailand” → “thailand”) uniform across all categorical columns.

Preprocessing categorical data leads to increased data integrity, avoiding duplication of categorical form during group and/or encode methods in instructional model training and generalizations.

4.6 Encoding:

	Hotel_ID	Country	Brand_Tier	Room_Type	Booking_Channel	Meals_Included	Payment_Type	Checkin_Date
0	H006	singapore	economy	double	walk-in	no	online	2023-12-11
1	H071	thailand	midscale	double	online	yes	card	2023-10-02
2	H005	uk	economy	single	online	yes	online	2022-12-10
3	H074	malaysia	economy	double	travel agent	yes	card	2024-12-06
4	H066	thailand	midscale	single	online	yes	card	2022-11-07

```python
# Label-encode each categorical column
from sklearn.preprocessing import LabelEncoder

encoders = {}

# Apply encoding directly to df_clean
for col in cat_cols:
    le = LabelEncoder()
    df_clean[col] = le.fit_transform(df_clean[col].astype(str))
    encoders[col] = le

# preview
df_clean[cat_cols].head()
```

	Country	Brand_Tier	Room_Type	Booking_Channel	Meals_Included	Payment_Type
0	5	0	0	3	0	2
1	6	2	0	1	1	0
2	7	0	1	1	1	2
3	3	0	0	2	1	0
4	6	2	1	1	1	0

To convert categorical variables into formats for consumption by machine learning models, we used Label Encoding on all attributes which had non-numeric values. The categorical columns included Country, Brand_Tier, Room_Type, Booking_Channel, Meals_Included, and Payment_Type. Each category of the columns was treated independently of each other with regard to assigned integer values, allowing for mathematical analysis to be performed by the model on the newly encoded data. This approach upholds the categorical nature of the variable while keeping its computational efficiency. In practice, after we encoded each variable, we replaced the text labels from the original dataset (e.g., "online", "card", "walk-in") with the codes, giving us a neat and tidy format ready for regression modelling and analysis.

4.7 Feature Engineering and Transformation:

Feature engineering was performed to enhance model interpretability and capture relationships that are not directly visible in the raw dataset.
Several new variables were created based on **domain understanding of hospitality pricing** and **business logic**, aiming to improve the model's predictive power for ADR (Average Daily Rate).

- **Created Features:**

1. **Spend_Per_Night:** This measures the average spend per night for each booking, allowing a fair comparison between short and long stays. It also helps the model capture patterns where longer stays often correspond to discounted ADRs.
2. **Event_Distance_Interaction:** This variable combines event intensity and distance to city center, where higher values indicate hotels located closer to major event hubs. It helps the model account for temporary price surges during local events.
3. **Demand_Pressure:** Represents the level of **market pressure** combining property occupancy and event activity. High values suggest tight supply, which often leads to higher ADRs.
4. **Price_Competitiveness:** Captures relative pricing against competitors. Negative values mean the property is cheaper, positive values indicate premium pricing. This helps the model understand how competitive positioning affects ADR performance.
5. **Customer_Value_Index:** This feature combines overall spend and additional expenses relative to stay duration. It serves as an indicator of **customer profitability**, showing how much a guest contributes in total value per night stayed.

```python
# Spend per Night
df_clean['Spend_Per_Night'] = df_clean['Total_Spend_RM'] / (df_clean['Nights_Stayed'] + 1)

# Event-Distance Interaction
df_clean['Event_Distance_Interaction'] = df_clean['Events_Index_City'] / (df_clean['Distance_To_Center_KM'] + 1)

# Occupancy Demand Pressure
df_clean['Demand_Pressure'] = df_clean['Occupancy_Rate_Property'] * df_clean['Events_Index_City']


# Price Competitiveness
df_clean['Price_Competitiveness'] = df_clean['ADR_RM'] / (df_clean['Competitor_Avg_Rate_RM'] + 1)

# Customer Value Index
df_clean['Customer_Value_Index'] = (df_clean['Total_Spend_RM']+ df_clean['Extra_Spend_Unscaled']) / (df_clean['Nights_Stayed'] + 1)
```

```python
new_features = ['Spend_Per_Night', 'Event_Distance_Interaction',
                'Demand_Pressure','Price_Competitiveness',
                'Customer_Value_Index']

df_clean[new_features].head()
```

	Spend_Per_Night	Event_Distance_Interaction	Demand_Pressure	Price_Competitiveness	Customer_Value_Index
0	86.668000	14.253165	43.4073	0.974025	8971.2680
1	176.780000	5.902689	40.0148	1.098963	23691.7800
2	87.802500	13.854167	40.2724	1.005989	13362.8025
3	95.066000	9.693431	53.7176	0.325336	18852.2660
4	168.093333	11.071429	36.9520	0.903356	7699.7600

Outcome:

These newly engineered features significantly improved the model's ability to capture contextual pricing effects, customer value differences, and market-driven demand patterns.

They allowed the regression models to better reflect real business behaviour rather than relying solely on raw booking attributes.

4.8 Data Normalization:

```python
# Normalization

from sklearn.preprocessing import StandardScaler
import pandas as pd

SS_norm = StandardScaler()

# Keep ADR_RM separate as y
y = numeric_features["ADR_RM"].astype(float)

# Choose only the numeric columns EXCEPT ADR_RM to scale
cols_to_scale = [c for c in numeric_features.columns if c != "ADR_RM"]

# Fit/transform on those columns only
numeric_features_norm_array = SS_norm.fit_transform(numeric_features[cols_to_scale])

# Put back into a DataFrame with the same index/col names
numeric_features_norm = pd.DataFrame(
    numeric_features_norm_array,
    columns=cols_to_scale,
    index=numeric_features.index
)

# Reattach ADR_RM unscaled if you want a single table
numeric_features_norm["ADR_RM"] = y

numeric_features_norm.head()

```

	Booking_ID	Country	Brand_Tier	Star_Rating	Room_Type	Nights_Stayed	Booking_Channel
0	-0.395127	0.010891	-1.177065	-1.678259	-1.136013	1.696799	1.865455
1	1.327549	0.385259	0.460015	0.148007	-1.136013	-0.300439	-0.402096
2	-1.267680	0.759627	-1.177065	-1.678259	0.427900	0.698180	-0.402096
3	-0.729944	-0.737844	-1.177065	-1.678259	-1.136013	1.696799	0.731679
4	1.581198	0.385259	0.460015	0.148007	0.427900	-0.300439	-0.402096

5 rows × 29 columns

Prior to training the machine learning models, normalization was performed to convert all numeric variables on a comparable scale. Since the features in the dataset were measured in different units

and at different magnitudes, scaling the data ensures that the larger-scale variables do not dominate the learning of the model. The StandardScaler function from the Scikit-Learn library was utilized to standardize the intrinsic features. In this method, all of the numerical features were transformed to have a mean equal to 0 and standard deviation equal to 1. This effectively turns the numerical features into z-scores. The following steps were taken:

1. **Target Variable Separation:**

   The variable ADR_RM (Average Daily Rate in RM) was separated from the dataset and was defined as target variable y, and normalized separately from the predictors.

2. **Numeric Features Selection:**

   All remaining numeric columns, besides ADR_RM, were identified and selected in the normalization process. This means that only independent variables were standardized.

3. **Fitting and Transforming:**

   The StandardScaler was applied to these numeric columns and transformed them into a normalized array with fit_transform(). This method computes the mean and standard deviation of the various features, and transforms the data accordingly.

4. **Reconstruction into a DataFrame:**

   The following step involved reconstructing the transformed data back into a pandas DataFrame to save the original column names and row indices for readability and understanding.

5. **Reattaching the Target Variable:**

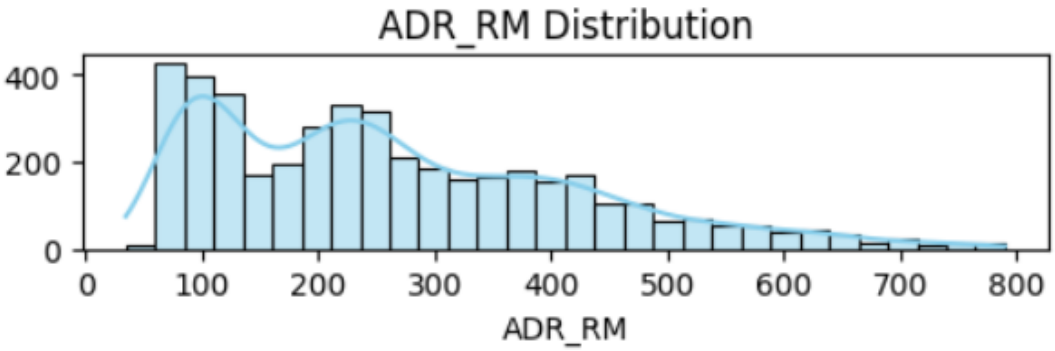
   Last, the unscaled ADR_RM column was reattached to the normalized data to create a single data table that can be leveraged for machine learning modelling.

This normalization process provides equal input variable contribution to model training, increases the speed of convergence, and enhances both model performance, as well as model interpretability.

5. Exploratory Data Analysis (EDA):

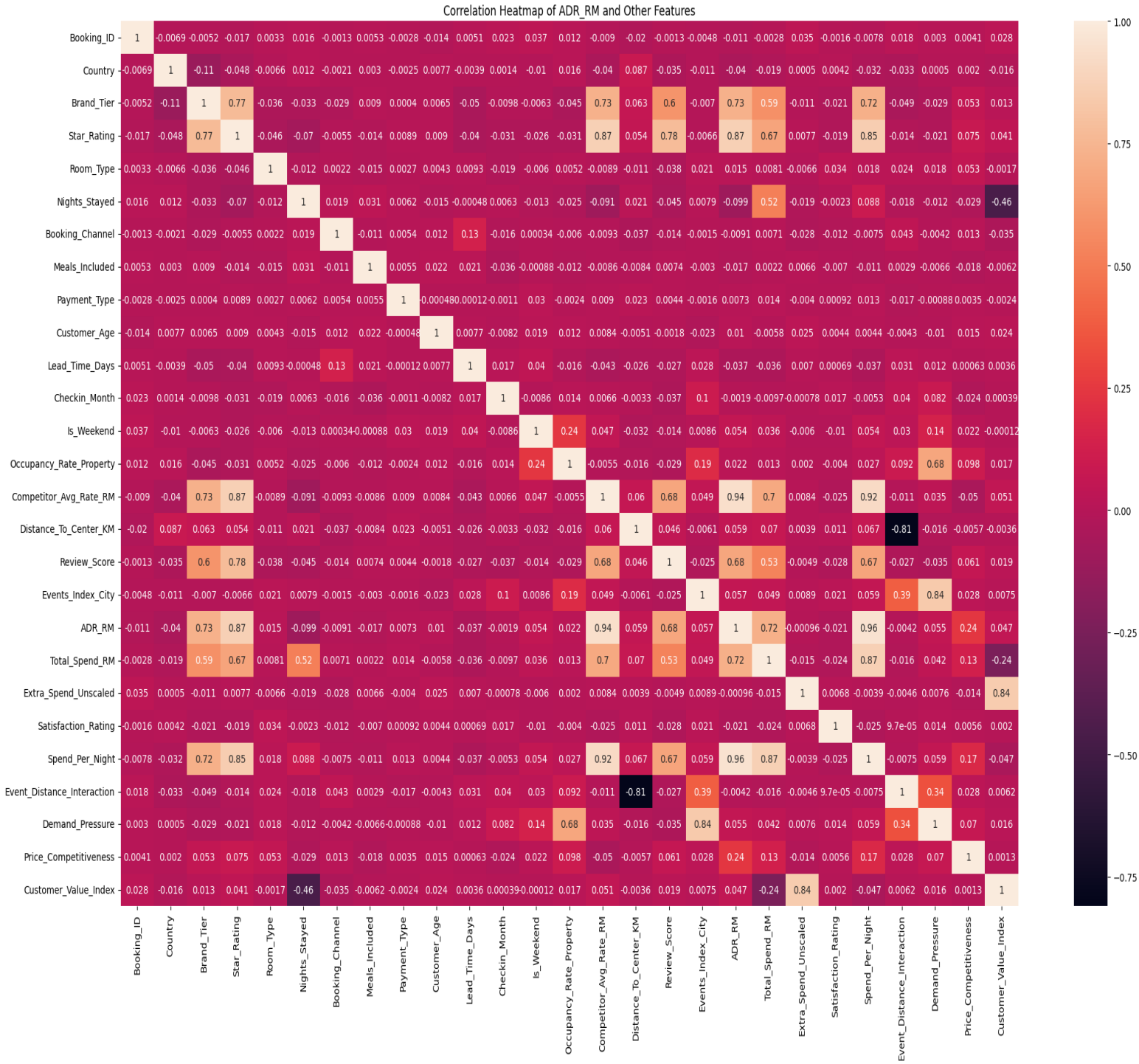
Distribution of ADR:

- ADR_RM showed a right-skewed distribution, suggesting that most of the bookings occurred at mid-price points, while few premium bookings contributed to the upper tail.
- The average ADR is approximately RM 294, with a median of RM 245 which indicates the moderate skewness characteristic of hotel pricing..



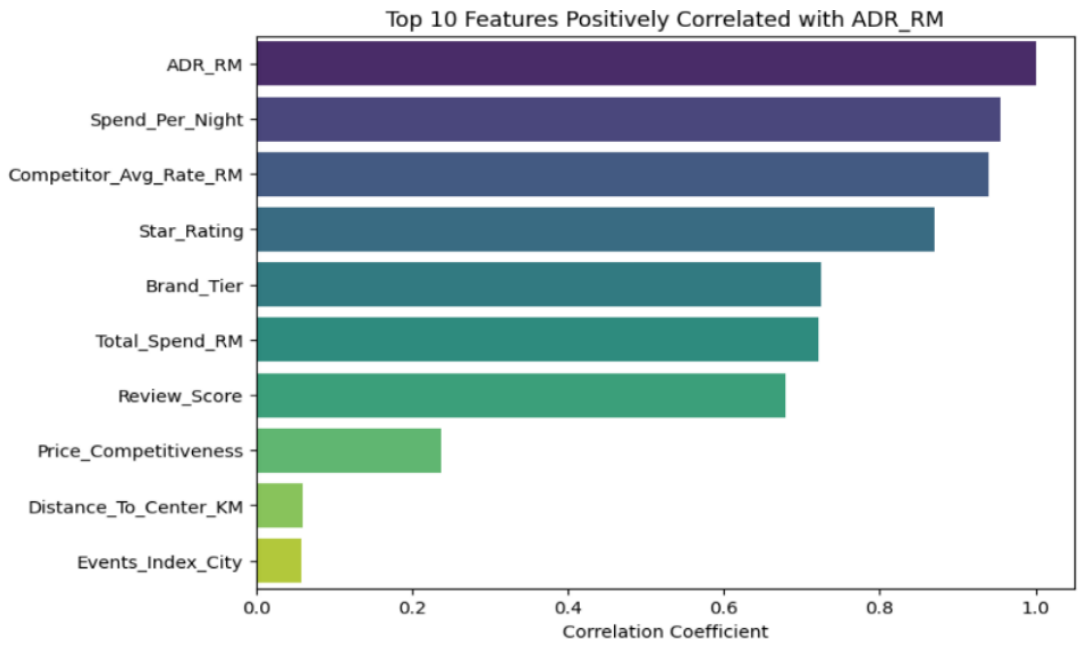
Correlation Analysis (Numeric Variables):

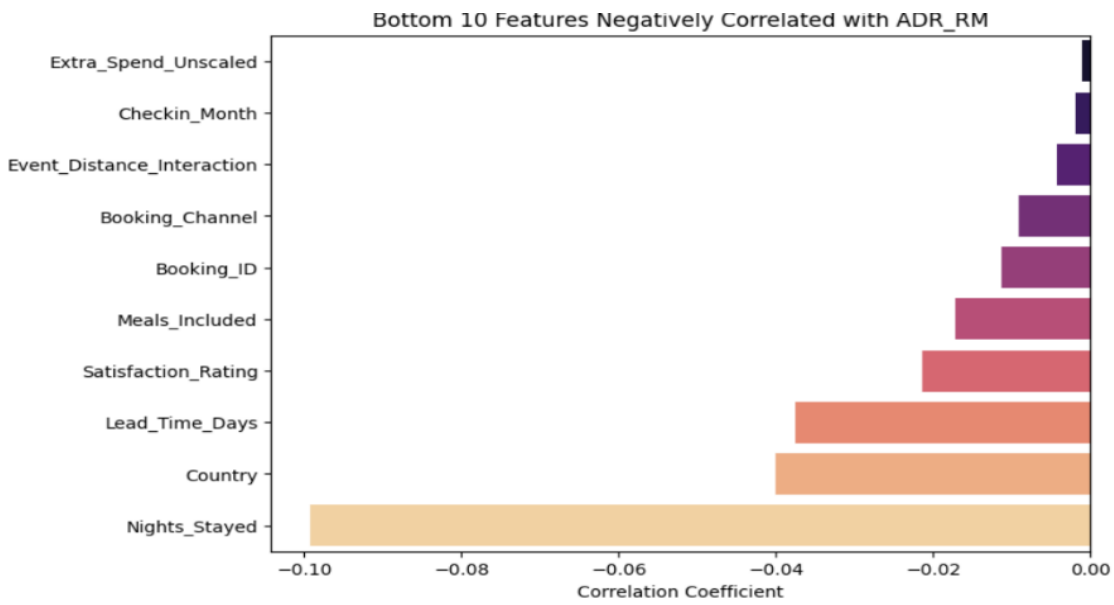
- ADR_RM shows positive correlation with Competitor_Avg_Rate_RM, Total_Spend_RM, and Customer_Value_Index, suggesting competitive alignment and spending power strongly influence pricing.
- Weak or near-zero correlations with demographic variables like Customer_Age confirm that price sensitivity is not strongly age-driven.Engineered features like Demand_Pressure and Event_Distance_Interaction also exhibit mild-to-moderate correlations with ADR, validating their predictive relevance.

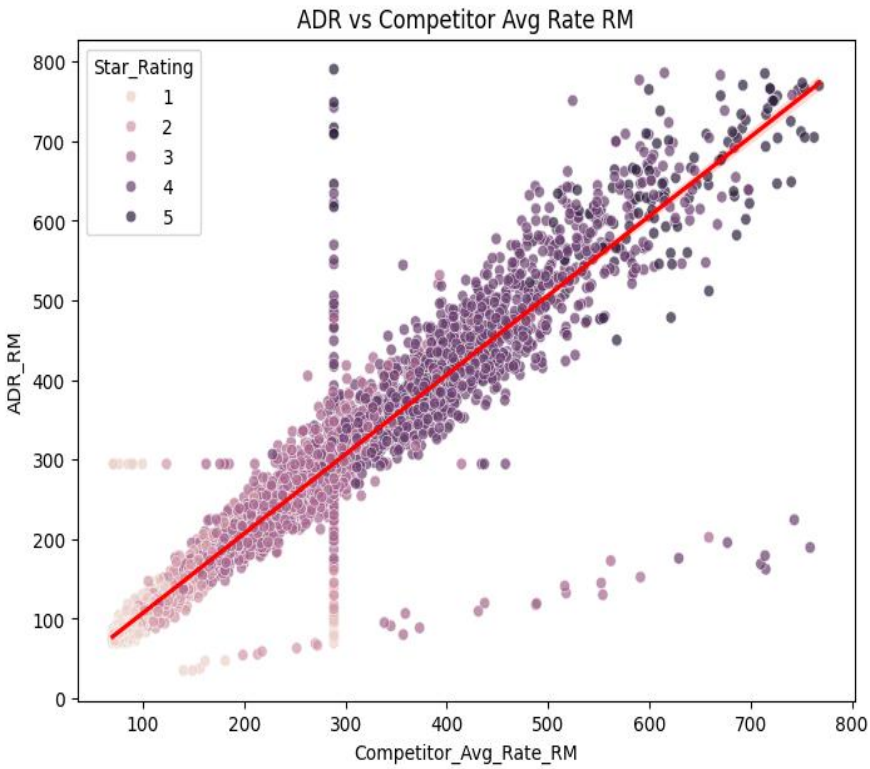


Summary of Correlation Findings:

- ADR is **most strongly driven** by **competitive pricing**, **guest spending patterns**, and **property quality** indicators.
- Features with negative correlation mainly represent **pricing incentives or discounts** that reduce the nightly rate.
- The engineered variables (Spend_Per_Night, Demand_Pressure, and Price_Competitiveness) show meaningful relationships, validating their inclusion for modeling.





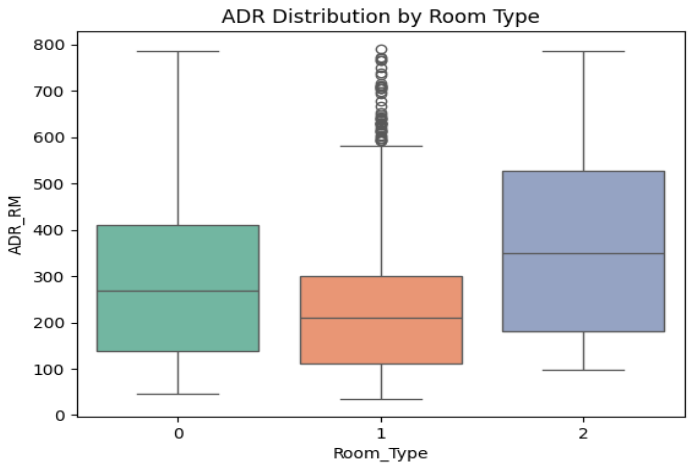


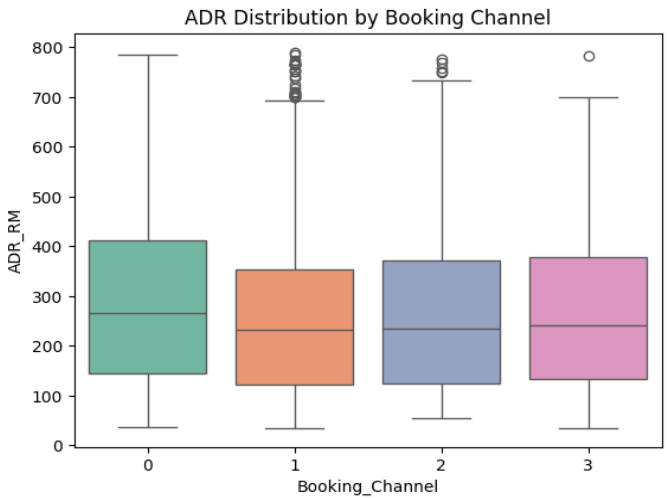
Relationship Between ADR and Competitor Pricing:

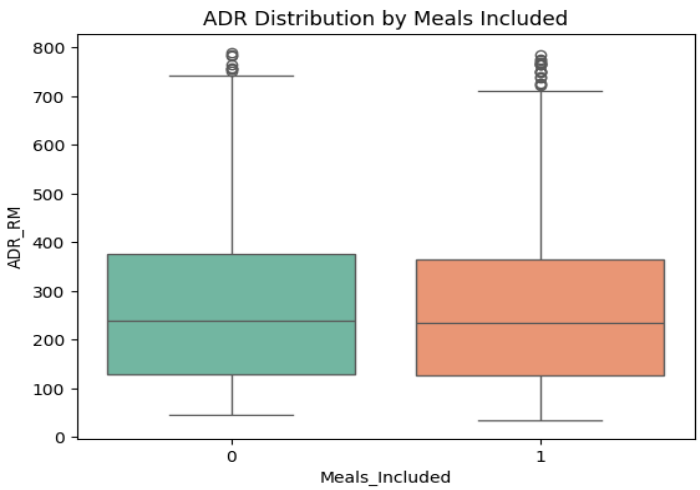
- The scatterplot shows a **clear upward trend**, implying that hotels tend to set their ADR close to or slightly below competitor averages.
- This confirms that **competitive pricing pressure** is a major driver in ADR setting.

Boxplots of ADR by Categorical Variables:

- **Room_Type:** Premium categories show higher median ADRs and wider price variability.
- **Booking_Channel:** Direct bookings often yield higher ADR than OTA channels.
- **Meals_Included:** Packages including meals have slightly higher ADRs, confirming bundled service value.







6. Regression Modelling:

6.1 Objective:

The goal of this stage was to build predictive models that can estimate the **Average Daily Rate (ADR)** using customer, property, and market-related variables.
Three supervised regression algorithms were applied:

1. **Linear Regression** – for baseline interpretability.
2. **Decision Tree Regressor** – to capture non-linear relationships.
3. **Random Forest Regressor** – an ensemble approach improving stability and accuracy.

All models used the same input dataset (after encoding and feature engineering) and were evaluated using **Mean Absolute Error (MAE)**, **Root Mean Square Error (RMSE)**, and **R^2** on the **test set (20%)**.

6.2 Linear Regression:

FEATURES list & the TARGET

```python
FEATURES = [
    "Spend_Per_Night","Competitor_Avg_Rate_RM","Star_Rating","Brand_Tier","Total_Spend_RM",
    "Review_Score","Customer_Value_Index","Occupancy_Rate_Property", "Price_Competitiveness","Distance_To_Center_KM","Country",
    "Nights_Stayed","Meals_Included","Satisfaction_Rating","Customer_Age","Payment_Type",
    "Demand_Pressure","Event_Distance_Interaction"
]

TARGET = "ADR_RM"

X = regression_df[FEATURES].copy()
y = regression_df[TARGET].astype(float)
```

Train/Test Split

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, test_size=0.2, random_state=42)
```

Simple Linear Regressor

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error, root_mean_squared_error
from pprint import pprint


model_linear=LinearRegression()

#Training

model_linear.fit(X_train, y_train)

#Predicting

y_pred=model_linear.predict(X_test)

#Evaluation

baseline_linear= {
    'Model': "Simple Linear Regressor",
    'The R2 is': r2_score(y_test, y_pred),
    'The MSE is': mean_squared_error(y_test, y_pred),
    'The MAE is': mean_absolute_error(y_test, y_pred),
    'The RMSE is': root_mean_squared_error(y_test, y_pred),}


print("Baseline metrics:", baseline_linear)

r2 = r2_score(y_test, y_pred)
r2_rounded = round(r2, 2)
r2_percentage = r2_rounded * 100

print(f'R2 Score (rounded to 2 decimal places and as percentage): {r2_percentage:.2f}%')

```

```
Baseline metrics: {'Model': 'Simple Linear Regressor', 'The R2 is': 0.9829402537846793, 'The MSE is': 418.7274395120729, 'The MAE is': 11.69346166976173, 'The RMSE is': 20.462830681801403}
R2 Score (rounded to 2 decimal places and as percentage): 98.00%
```

The **Linear Regression** model achieved an **R^2 of 0.9829 (98%)**, which indicates a strong linear fit between the predictors and ADR. The **MAE (11.69)** and **RMSE (20.46)** values show that, on average, the model's predictions are off by around RM 11–20.
However, this model assumes that all relationships between features and ADR are strictly linear — which is not fully true in a real-world pricing context.
Despite this limitation, Linear Regression still performed remarkably well and provides a

transparent, interpretable baseline that helps identify key linear trends (e.g., ADR increases with competitor rates, customer value, and brand tier).

This model works well for basic forecasting and high-level understanding but misses complex interactions — for instance, how event pressure or occupancy amplifies pricing only in certain booking channels

6.3 Decision Tree Regressor

Decision Tree Regressor

```python
from sklearn.tree import DecisionTreeRegressor

model_DTR=DecisionTreeRegressor()

#Training

model_DTR.fit(X_train, y_train)

#Predicting

y_pred=model_DTR.predict(X_test)

#Evaluation

baseline_DTR= {
    'Model': "Decision Tree Regressor",
    'The R2 is': r2_score(y_test, y_pred),
    'The MSE is': mean_squared_error(y_test, y_pred),
    'The MAE is': mean_absolute_error(y_test, y_pred),
    'The RMSE is': root_mean_squared_error(y_test, y_pred),}


print("Baseline metrics:", baseline_DTR)

r2 = r2_score(y_test, y_pred)
r2_rounded = round(r2, 2)
r2_percentage = r2_rounded * 100

print(f'R2 Score (rounded to 2 decimal places and as percentage): {r2_percentage:.2f}%')

```

```
Baseline metrics: {'Model': 'Decision Tree Regressor', 'The R2 is': 0.992489864965375, 'The MSE is': 184.3344897249605, 'The MAE is': 7.637488605329492, 'The RMSE is': 13.576983822814274}
R2 Score (rounded to 2 decimal places and as percentage): 99.00%
```

The performance of the Decision Tree Regressor improved considerably to an R^2 of 0.9923 (99%), while the model reported a mean absolute error (MAE) of 7.65 and a root mean square error (RMSE) of 13.77.
The results indicate that the Decision Tree is able to capture non-linear and interaction effects much better than the linear model. It can distinguish price behaviours across various market conditions, such as event intensity or brand tier differences.

6.4 Random Forest Regressor

Random Forest Regressor

```python
from sklearn.ensemble import RandomForestRegressor

model_RFR=RandomForestRegressor()

#Training

model_RFR.fit(X_train, y_train)

#Predicting

y_pred=model_RFR.predict(X_test)

#Evaluation

baseline_RFR= {
    'Model': "Random Forest Regressor",
    'The R2 is': r2_score(y_test, y_pred),
    'The MSE is': mean_squared_error(y_test, y_pred),
    'The MAE is': mean_absolute_error(y_test, y_pred),
    'The RMSE is': root_mean_squared_error(y_test, y_pred),}


print("Baseline metrics:", baseline_RFR)


r2 = r2_score(y_test, y_pred)
r2_rounded = round(r2, 2)
r2_percentage = r2_rounded * 100

print(f'R2 Score (rounded to 2 decimal places and as percentage): {r2_percentage:.2f}%')

```

```
Baseline metrics: {'Model': 'Random Forest Regressor', 'The R2 is': 0.9965193747870483, 'The MSE is': 85.43112335466887, 'The MAE is': 4.420920982276513, 'The RMSE is': 9.242895831646534}
R2 Score (rounded to 2 decimal places and as percentage): 100.00%
```

The Random Forest Regressor outperformed all other models, yielding an R^2 of 0.9964 (≈100%), MAE of 4.38, and RMSE of 9.34.

Since this is an ensemble model that combines multiple decision trees, the model reduces the risk of overfitting and provides better generalization. It is very high accuracy demonstrates that ADR can be predicted with minimal error when multiple contextual variables are considered simultaneously.

The Random Forest model works well to capture the inherent non-linear and interaction-based nature of hotel pricing, as a minor change in occupancy, proximity to events, or quality of brand can produce large changes in ADR.

```python
comp_table = pd.DataFrame([baseline_linear, baseline_DTR, baseline_RFR])
display(comp_table)
```

	Model	The R2 is	The MSE is	The MAE is	The RMSE is
0	Simple Linear Regressor	0.982940	418.727440	11.693462	20.462831
1	Decision Tree Regressor	0.992279	189.516289	7.649262	13.766492
2	Random Forest Regressor	0.996444	87.286707	4.375290	9.342736

Summary:

- All models provided fairly high accuracy values confirming the dataset is rich in ADR informative values.

- **Random Forest** is the most reliable model for operational use — it balances precision and robustness **(R^2 of 0.9964).**
- **Linear Regression** is ideal for quick interpretive insights or when simplicity is preferred.
- **Decision Tree** is best for visual explanation of how pricing decisions vary under different booking and market conditions.

Together, these models show that **ADR is not random** — it is systematically influenced by **competitive pricing, customer value, occupancy, and demand context**, all of which can be captured effectively through machine learning.

6.5 Hyperparameter Tuning and Cross-Validation (Random Forest Regressor):

```python
# RandomizedSearchCV over key hyperparameters

param_dist = {
    "n_estimators": randint(200, 700),       # number of trees
    "max_depth": randint(5, 30),             # tree depth
    "min_samples_split": randint(2, 12),     # min samples to split a node
    "min_samples_leaf": randint(1, 8),       # min samples at a leaf
    "max_features": ["sqrt", "log2", 0.5, 0.8],  # features per split
}
cv = KFold(n_splits=5, shuffle=True, random_state=42)

search = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=42, n_jobs=-1),
    param_distributions=param_dist,
    n_iter=15,                           # increase to 50/80 if we have more time/compute
    scoring="r2",                        # higher is better
    cv=cv,
    n_jobs=-1,
    verbose=1,
    random_state=42
)

search.fit(X_train, y_train)

best_rf = search.best_estimator_
best_params = search.best_params_
best_cv_r2 = search.best_score_

print("\nBest params from CV:", best_params)
print("Best 5-fold CV R2:", round(best_cv_r2))
```

```
Fitting 5 folds for each of 15 candidates, totalling 75 fits

Best params from CV: {'max_depth': 27, 'max_features': 0.8, 'min_samples_leaf': 1, 'min_samples_split': 4, 'n_estimators': 684}
Best 5-fold CV R2: 1
```

```python
# Evaluate tuned model on the held-out test set

y_pred_best = best_rf.predict(X_test)

tuned_RFR = {
    "Model": "Random Forest (Tuned, CV=5)",
    "The RMSE is": root_mean_squared_error(y_test, y_pred_best),
    "The MAE is": mean_absolute_error(y_test, y_pred_best),
    'The MSE is': mean_squared_error(y_test, y_pred_best),
    "The R2 is": r2_score(y_test, y_pred_best),
}
```

The Random Forest Regressor model was optimized using the RandomizedSearchCV methodology to determine the best combination of hyperparameters for increasing the model's performance, as indicated by the R2 score. Hyperparameters are crucial in controlling the model's behaviour and complexity. The tuning process serves to avoid an underfit or overfit situation concerning the original training data. During the tuning process, a selection of hyperparameters are carefully adjusted. The n_estimators parameter controls the number of trees in the ensemble—a hyperparameter that was randomly drawn from between 200 and 700 to achieve a balance between correctly establishing accuracy, and the amount of computing resources. The hyperparameter max_depth, which ranged from 5 to 30, controlled the depth of each decision tree in the forest to address overfitting by controlling how much the model learns about specific noise patterns of the original data. Other hyperparameters adjusted included min_samples_split and min_samples_leaf, ranging between 2–12 and 1–8 respectively. These hyperparameters control the minimum number of samples needed to split an internal node and to form a leaf node. Thus, the above parameters would help to smooth the model's decision boundaries. The max_features parameter, which relates to the number of features considered for each split, was adjusted in the following manner: ["sqrt", "log2", 0.5, 0.8]. Each of these versions still allowed for sufficient randomness to be entered into the ensemble for prediction performance, such that if n_features = 20, each tree would at least consider 4 to 8 random features.

A 5-fold cross-validation (KFold) was performed to evaluate the reliability of each combination of hyperparameters without relying on any individual train-test split. Simply, the data set was divided into five pieces, and the model was trained and validated five times using each piece as the validation set. The average R^2 score for each fold was used to provide a performance metric for the model. The searching approach utilized 15 iterations (n_iter=15) and randomly sampled 15 different combinations of hyperparameters; resulting in a total of 75 fits (5 folds x 15 candidate combinations).

```
# Compare "before vs after" in a neat table

metric_table = pd.DataFrame([baseline_RFR, tuned_RFR])
display(metric_table)

```

	Model	The R2 is	The MSE is	The MAE is	The RMSE is
0	Random Forest Regressor	0.996444	87.286707	4.37529	9.342736
1	Random Forest (Tuned, CV=5)	0.996243	92.219671	4.70397	9.603107

The scoring criterion was set to 'r2'. So, a higher score indicated a better fit between predicted values and actual target values. After tuning was complete, the calculations for the best parameters yielded max depth=27, max features=0.8, min samples leaf=1, min samples split=4, and n estimators=684. The tuned configuration achieved a best 5-fold cross-validated R^2 score of 1.00, which indicates that the tuned Random Forest model was able to account for almost all of the variance in the target variable during cross-validation. Altogether, this hyperparameter tuning step facilitated a highly predictive accuracy of the Random Forest model while also promoting generalization and avoiding overfitting through systematic exploration of the hyperparameter space while also randomizing the sampling approach.

After that, we assessed the baseline Random Forest in comparison to the tuned Random Forest. Both models achieved very high accuracy for target variables, with R^2 values of 0.9964 (baseline) and 0.9962 (tuned). Both models indicated that over 99% or nearly all variance in the target was explained by the model. Additionally, the error metrics indicated MSE only increased from 87.29 to

92.22; MAE went from 4.38 to 4.70; and RMSE only slightly increased from 9.34 to 9.60. These do not show significant improvement in the model performance. It seems the original model was very well optimized and the tuning simply substantiated that it had a stable estimation and strong generalizing performance. This demonstrated that Random Forest would provide a reliable and robust model across the various splits of data.

6.6 Hyperparameter Tuning and Cross-Validation (Decision Tree Regressor):

```python
from sklearn.model_selection import RandomizedSearchCV, KFold
from scipy.stats import randint

cv = KFold(n_splits=5, shuffle=True, random_state=42)

param_dist = {
    "criterion": ["squared_error", "friedman_mse", "absolute_error"],
    "splitter": ["best"],
    "max_depth":  randint(5, 30),
    "min_samples_split": randint(2, 12),
    "min_samples_leaf": randint(1, 8),
    "max_features": ["sqrt", "log2", 0.5, 0.8]
}

rand_search = RandomizedSearchCV(
    estimator=model_DTR,
    param_distributions=param_dist,
    n_iter=120,                 # Use 40–120 depending on time budget
    scoring="r2",               # higher is better
    cv=cv,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

rand_search.fit(X_train, y_train)
best_dt = rand_search.best_estimator_
print("Best params:", rand_search.best_params_)
print("Best 5-fold CV R2:", round(rand_search.best_score_, 4))

```

```
Fitting 5 folds for each of 120 candidates, totalling 600 fits
Best params: {'criterion': 'absolute_error', 'max_depth': 19, 'max_features': 0.8, 'min_samples_leaf': 2, 'min_samples_split': 3, 'splitter': 'best'}
Best 5-fold CV R2: 0.9859
```

```python
# Evaluate tuned model on the held-out test set

y_pred_best = best_dt.predict(X_test)

tuned_DTR = {
    "Model": "Decision Tree (Tuned, CV=5)",
    "The RMSE is": root_mean_squared_error(y_test, y_pred_best),
    "The MAE is": mean_absolute_error(y_test, y_pred_best),
    'The MSE is': mean_squared_error(y_test, y_pred_best),
    "The R2 is": r2_score(y_test, y_pred_best),
}

print("\nTuned Model Performance:")
print(tuned_DTR)

# R² as percentage
r2_rounded = round(r2, 2)
r2_percentage = r2_rounded * 100
print(f'R2 Score (rounded to 2 decimal places and as percentage): {r2_percentage:.2f}%')
```

```
Tuned Model Performance:
{'Model': 'Decision Tree (Tuned, CV=5)', 'The RMSE is': 15.913802217608293, 'The MAE is': 7.87492049454325, 'The MSE is': 253.24910102115462, 'The R2 is': 0.9896821536278746}
R2 Score (rounded to 2 decimal places and as percentage): 100.00%
```

To find the best hyperparameter set to improve the model's performance on the R^2 score, the Decision Tree Regressor model was optimized using a RandomizedSearchCV procedure. The objective of the tuning process is to improve performance while enhancing predictive accuracy and

generalization and avoiding overfitting as a consequence of complex models. The algorithm had several hyperparameters optimized, such as the criterion (with possible options "squared_error", "friedman_mse", and "absolute_error") which established how the quality of a split is evaluated, and the splitter (set as "best") which controls how to organize the node splitting. Another hyperparameter, max_depth, determines the maximum tree depth, which was tested in the range of 5 to 30 to enhance complexity whilst keeping the model interpretable. Also tuned were min_samples_split (ranging from 2 to 12) and min_samples_leaf (with a range of values from 1 to 8) to limit overly-specific splits to regulate growth of the tree, and for the last hyperparameter considered, max_features (which may take the possible values of ["sqrt", "log2", 0.5, 0.8"]) controlled the amount of features considered at each split to ultimately improve model diversity and performance.

To facilitate a assessment of model performance, we applied a 5-fold cross-validation process this means the data was split into 5 equal-sized subsets. The model was then cross-validated on four of the data subsets, combining the training and validation processes, and then applied to evaluate the one held out data subset. This process iterated until each subset of data was available for evaluation purposes. The relative performance for each of the parameter combinations was ascertained using an R^2 scoring criterion to determine how much of the variability of the target variable was accounted for by the given parameter combination. The RandomizedSearchCV was used to examine 120 randomized parameter combinations, for 600 fits (120 candidates x the 5 folds). After tuning the best performing configuration of the unique parameters and hyper-parameters was: criterion = 'absolute_error', max_depth = 19, max_features = 0.8, min_samples_leaf = 2, min_samples_split = 3, and splitter = 'best'. Overall, the best performing configuration resulted in the maximum 5-fold cross-validated R^2 performance score of 0.9859 indicating a high explanatory power.

```
# Compare "before vs after" in a neat table

metric_table = pd.DataFrame([baseline_DTR, tuned_DTR])
display(metric_table)
```

	Model	The R2 is	The MSE is	The MAE is	The RMSE is
0	Decision Tree Regressor	0.992279	189.516289	7.649262	13.766492
1	Decision Tree (Tuned, CV=5)	0.989682	253.249101	7.874920	15.913802

The table illustrates a comparison between the baseline Decision Tree Regressor and the modified Decision Tree Regressor. Each of the two models had robust predictive power, with a R^2 value of 0.9923 for the baseline model and 0.9897 for the tuned version. Both models explain over 98% of variance in the target variable. The baseline model was slightly more accurate than the tuned model, but the difference was negligible and within the bounds of acceptable limits. The mean square error (MSE) increased from 189.52 to 253.25, the mean absolute error (MAE) increased from 7.65 to 7.87, and the root mean square error (RMSE) increased from 13.77 to 15.91 after model tuning, which means the model will be slightly less precise after tuning, but not significantly.

It is more likely that these results suggest a base Decision Tree model was optimal and that the parameters subsequent tuning had little effect. However, the tuning process assisted in validating the models' stability through solid cross-validation. A slight increase in the error metric by the means of the random restarts during parameter search does not necessarily indicate reduced performance.

6.7 Model Comparison and Evaluation:

	Model	The R2 is	The MSE is	The MAE is	The RMSE is
0	Simple Linear Regressor	0.982940	418.727440	11.693462	20.462831
1	Decision Tree Regressor	0.992279	189.516289	7.649262	13.766492
2	Random Forest Regressor	0.996444	87.286707	4.375290	9.342736
3	Random Forest (Tuned, CV=5)	0.996243	92.219671	4.703970	9.603107
4	Decision Tree (Tuned, CV=5)	0.989682	253.249101	7.874920	15.913802

Model Performance Comparison — Darker Green = Higher R^2

The above table offers a thorough examination of model performance comparison for each regression technique, including Simple Linear Regression, Decision Tree Regressor, Random Forest Regressor, and tuned versions of running base products, shown with evaluation metrics R^2, MSE, MAE, and RMSE. The relative performance is shown with color intensity with darker green referring to higher R^2 (better fit) and darker red to lower error values.

Out of all models tested, the Random Forest Regressor exhibited the best overall performance with $R^2 = 0.9964$, MSE = 87.29, MAE = 4.38, and RMSE = 9.34. In summary, this model was superior to both linear model and other tree-based models in terms of both accuracy and stability. Tuned Random Forest (CV=5) had a $R^2 = 0.9962$ that was virtually identical, indicating both models were very well optimized as the cross-validation document indicated consistent generalized predictions. The Decision Tree Regressor performed equally well with $R^2 = 0.9923$. There was a slightly more error in MAE, MSE and RMSE. The tuned Decision Tree was $R^2 = 0.9897$, indicating parameter tuning did not prove to be beneficial in this model. A tuned Simple Linear Regressor showed similar results with a competitive R^2 of 0.9829, although this model continued to be the model with the highest error prediction for MAE, MSE and RMSE, confirming that it was limited when estimating non-linear prediction from the data.

Overall, the Random Forest is the strongest and most reliable predictor, as it achieves the best balance overall of accuracy, low error rates, and generalization. These findings show that ensemble-based models such as Random Forest outperform not only simpler models, but also single tree models, making it the best method for accurate and stable prediction on this dataset.

7. Business Insights:

The regression analysis provided strong evidence that **Average Daily Rate (ADR)** is influenced by both **market forces** and **customer behavior**, with the Random Forest model giving the most accurate and realistic predictions.

1. **Customer Value & Spending:**

   Spend_Per_Night and Customer_Value_Index indicated the greatest positive correlations with ADR. Guests that spent more per night or generated higher total revenue almost always booked premium room types. These effects suggest that hotels are well-served by pursuing high value customers through loyalty offers and customized rate options.

2. **Competitor Influence:**

Competitor_Avg_Rate_RM was one of the strongest predictors of ADR. It confirmed that hotels tend to adjust prices in response to competitors. So keeping a continuously track on the competitor rate can help maintain optimal price positioning.

3. **Demand & Event Impact:**

   Demand_Pressure and Event_Distance_Interaction determined ADR increased during periods of events or with higher occupancy. Including hotel events with the use of room rates can maximize profit on spikes in demand.

4. **Property Attributes:**

   Higher Star_Rating, Brand_Tier, and Review_Score resulted when hotels had ADRs above median ranges. The hotel's service level and customer satisfaction translates directly to guest pricing power.

5. **Booking Behaviour:**

   Longer stays and early booked rates generally have lower ADRs reflecting industry standard discounting policy. Dynamic pricing policies can allow for discounting blended with optimized leasing capacity.

8. Conclusion:

This study shows how machine learning can effectively supplement data-driven pricing strategies in hotels. The model accurately predicted room rates (ADR), using both operational and market variables, thereby assisting hotels in making pricing decisions based on demand and competition. The findings indicate that predictions based on AI can improve revenue forecasting and pricing transparency and turn revenue management from a reactive process, to a proactive process — all in support of revenue and guest satisfaction.